Project Report On:

Ayush-Guard : Clinical Decision Support System

Submitted in partial fulfillment of the requirements

For

B.Tech in Information Technology

By SY B.Tech IT **Batch C Group 1**

Aditya Shimoga - 241080067

Taha Valiji - 241080071

Awwab Wadekar - 241080077

Yash Wadhwa - 241080078

Under the guidance of Prof. Saksham Kataria

Department of Computer Engineering & Information Technology

Veermata Jijabai Technological Institute

Matunga, Mumbai - 400019

Abstract

Ayush-Guard is a clinical decision support system designed to help pharmacists check whether a medicine is safe for a patient before dispensing it. The platform resolves brand names into generic salts, compares them against the patient's active medicines and health conditions, and then returns safety alerts or safer alternatives. The project combines a React-based frontend, a FastAPI backend, and a Supabase-hosted PostgreSQL database to deliver a structured and reliable workflow for retail pharmacy use.

---

1. Problem Statement

The current process of dispensing medicines at local pharmacies operates with a critical and largely unaddressed gap: the absence of a real-time clinical safety check at the point of dispensing. In routine pharmacy practice, a pharmacist is frequently presented with a brand name prescription, yet the true clinical safety of that medicine is governed not by its commercial identity, but by its active pharmaceutical ingredient — the generic salt — and how that ingredient interacts with the patient's existing pharmacological profile. A comprehensive safety assessment must account for the patient's concurrent medications, diagnosed medical conditions, known allergies, and, where applicable, hereditary pharmacogenomic sensitivities. In a typical pharmacy environment, this constellation of information is rarely consolidated in a single accessible record, leaving pharmacists to make dispensing decisions under conditions of significant informational constraint.

This systemic deficiency gives rise to serious and preventable clinical risks. Harmful drug-drug interactions may go undetected when two medicines sharing overlapping mechanisms or metabolic pathways are dispensed concurrently. Contraindicated medicines may be issued to patients whose underlying conditions make such therapy clinically inappropriate. Duplicate therapy — wherein two drugs of the same class or with the same active ingredient are co-administered — may result in unintended overdose or therapeutic failure. Collectively, these lapses contribute to a broader burden of avoidable adverse drug events, which carry significant consequences for patient health outcomes, healthcare expenditure, and public trust in the pharmaceutical dispensing system.

This challenge is felt most acutely in rural and semi-urban communities, where the consequences of inadequate pharmaceutical oversight are disproportionately severe. In these settings, patients frequently lack access to specialist medical care, and the local pharmacist often serves as the first — and sometimes only — point of healthcare contact. Patients in rural areas are more likely to be managing multiple chronic conditions without regular physician follow-up, and are less likely to have a consolidated medical record accessible to the dispensing pharmacist. Furthermore, limited health literacy and geographic barriers to secondary care mean that an adverse drug event that might be quickly identified and managed in an urban clinical setting can escalate into a serious or life-threatening episode before appropriate intervention is possible. The absence of a robust clinical safety check at the rural pharmacy counter is therefore not merely a procedural shortcoming — it is a significant public health vulnerability.

2. Project Domain and Objectives

This project lies at the intersection of health informatics, database management, and web application engineering. Its purpose is not only to store medical data, but to process it at the point of care and turn it into a useful decision-making tool.

- To identify the actual generic salt behind a medicine brand name.
- To compare the selected medicine against current medications and health conditions.
- To warn the pharmacist about severe, moderate, or low-risk situations.
- To suggest a safer therapeutic substitute when a high-risk conflict is detected.
- To maintain structured access control and database integrity through Supabase and relational constraints.

---

3. Proposed Solution

3.1 The Clinical Gap:

"Blind" Point-of-Sale Dispensing Currently, the retail pharmacy sector in India operates in a clinical vacuum. Pharmacists dispense medications—both prescribed and over-the-counter—based entirely on isolated transactions, completely disconnected from a patient's longitudinal medical history. This lack of real-time clinical context leads to preventable Adverse Drug Events (ADEs), specifically dangerous drug-to-disease contraindications and overlapping drug-drug interactions. Without a localized safety layer, life-threatening prescribing errors or patient-driven double-dosing go entirely undetected at the retail counter.

3.2 The Objective:

Ayush-Guard The objective of this project is to architect "Ayush-Guard," a point-of-sale Clinical Decision Support System (CDSS). The system is designed to intercept the pharmacy transaction, contextualize the requested medication against the patient's authenticated digital health profile, and output an immediate risk assessment before the medication is physically dispensed.

3.3 Core Functional Requirements:

To solve the informational disconnect at the pharmacy counter, the system is designed to fulfill three strict functional objectives:

- Commercial-to-Chemical Entity Resolution: The system must be capable of taking highly variable, unstructured commercial brand names input by a pharmacist and accurately abstracting them into their standardized generic chemical salts. This is required to evaluate clinical risk, which is tied to the chemical, not the brand.
- Automated Contraindication Screening: The system must perform an instantaneous cross-reference between the resolved chemical salt and the patient's active medical profile (past prescriptions and diagnosed chronic conditions) to detect any clinical conflicts.
- Risk Stratification & Therapeutic Substitution: The system must classify the identified risks by clinical severity (Critical/Moderate/Safe) to provide immediate visual guidance.

---

4. Tech Stack

Component	Technology Used
Frontend	React, Vite, CSS, Tailwind
Backend	Python, FastAPI, APIRouter
Database	PostgreSQL via Supabase
Data Validation	Pydantic
Testing	Pytest
Deployment	Vercel, Render, Supabase

---

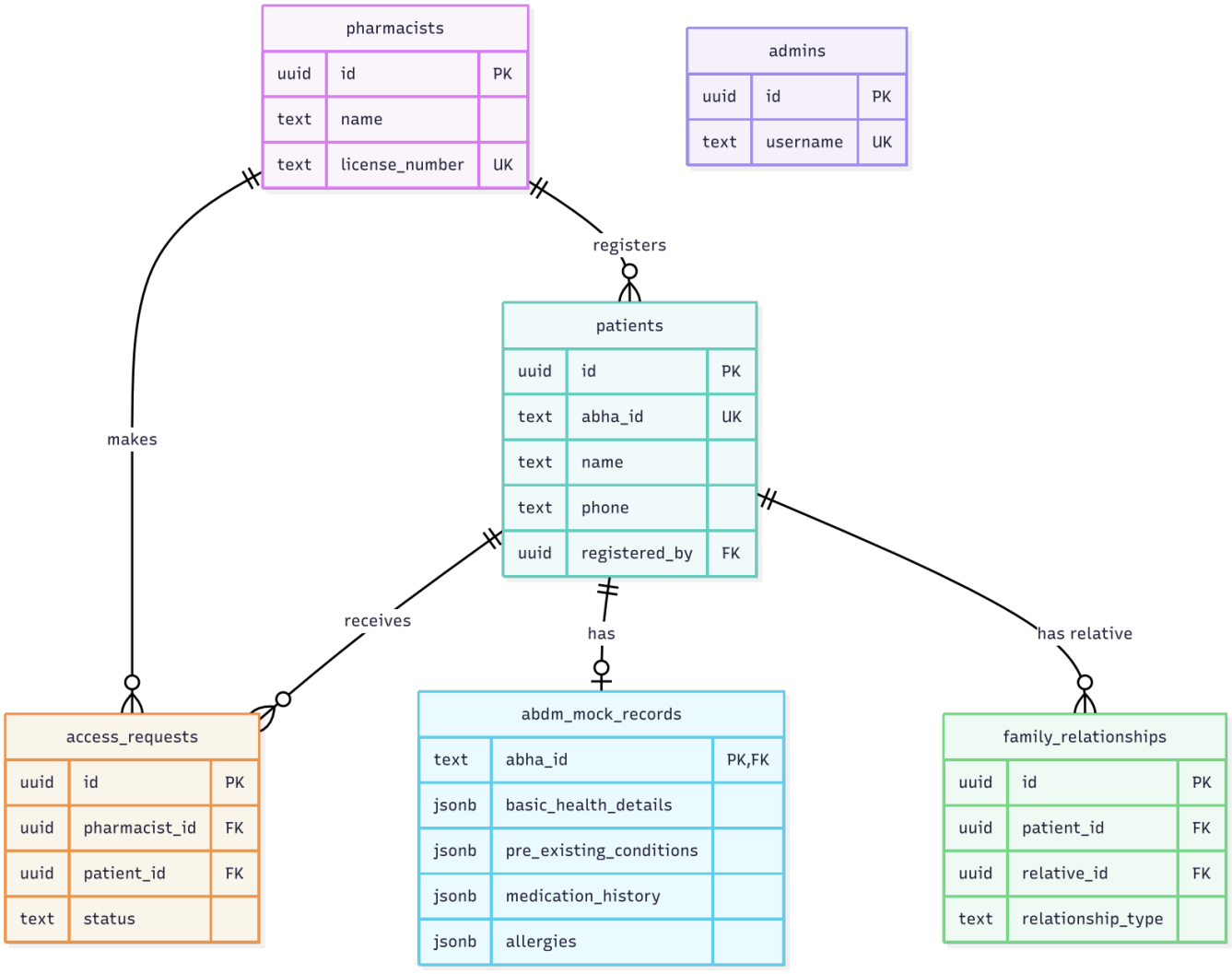
5. Database Schema

We use Supabase (PostgreSQL) as our primary data layer. The system utilizes structured relational tables alongside flexible JSONB columns to mock the external FHIR-like records.

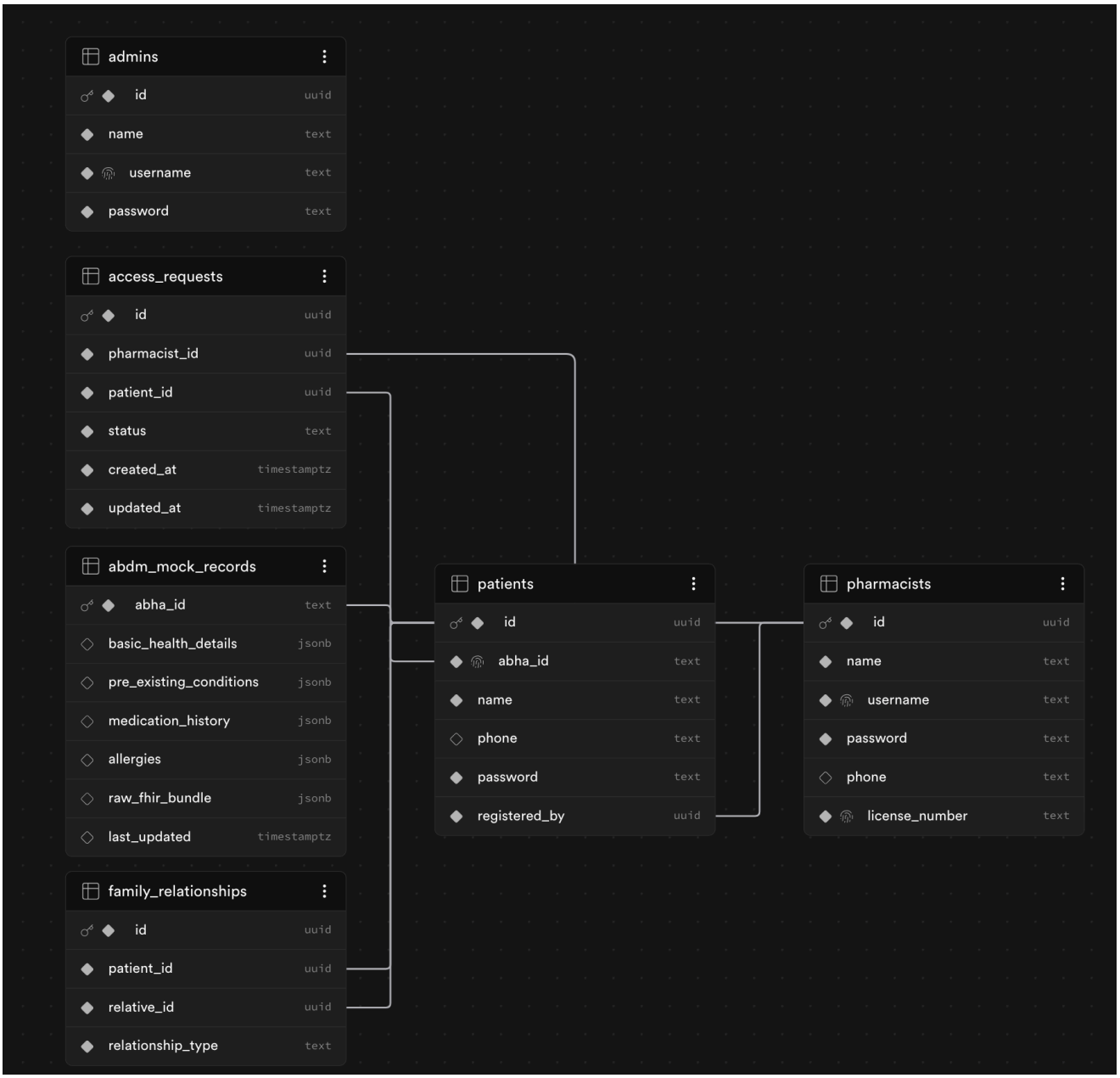
- **patients:** Stores structural identity data (name, abha_id). No medical history is stored directly here to maintain separation of concerns.
- **abdm_mock_records:** A JSONB representation of the external ABDM gateway. It holds arrays of medication_history, pre_existing_conditions, allergies, and raw_fhir_bundles.
- **access_requests:** Connects patients to pharmacists, representing the digital consent framework (PENDING, ACCEPTED, REJECTED states).
- **family_relationships:** Maps recursive patient IDs (e.g., Father to Son) to actively propagate genetic sensitivity warnings (e.g., G6PD Deficiency inheritance).
- **pharmacists & admins:** Stores credentialing and license information for registered healthcare vendors.

```
CREATE TABLE public.abdm_mock_records (
  abha_id text NOT NULL,
  basic_health_details jsonb DEFAULT '{}'::jsonb,
  pre_existing_conditions jsonb DEFAULT '[]'::jsonb,
  medication_history jsonb DEFAULT '[]'::jsonb,
  allergies jsonb DEFAULT '[]'::jsonb,
  raw_fhir_bundle jsonb,
  last_updated timestamp with time zone DEFAULT timezone('utc'::text, now()),
  CONSTRAINT abdm_mock_records_pkey PRIMARY KEY (abha_id),
      CONSTRAINT abdm_mock_records_abha_id_fkey FOREIGN KEY (abha_id) REFERENCES public.patients(abha_id)
);
CREATE TABLE public.access_requests (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  pharmacist_id uuid NOT NULL,
  patient_id uuid NOT NULL,
  status text NOT NULL DEFAULT 'PENDING'::text,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  updated_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT access_requests_pkey PRIMARY KEY (id),
    CONSTRAINT access_requests_pharmacist_fkey FOREIGN KEY (pharmacist_id) REFERENCES public.pharmacists(id),
      CONSTRAINT access_requests_patient_fkey FOREIGN KEY (patient_id) REFERENCES public.patients(id)
);
CREATE TABLE public.admins (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  name text NOT NULL,
  username text NOT NULL UNIQUE,
  password text NOT NULL,
  CONSTRAINT admins_pkey PRIMARY KEY (id)
);
CREATE TABLE public.family_relationships (
```

```
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  patient_id uuid NOT NULL,
  relative_id uuid NOT NULL,
  relationship_type text NOT NULL,
  CONSTRAINT family_relationships_pkey PRIMARY KEY (id),
    CONSTRAINT family_relationships_patient_fkey FOREIGN KEY (patient_id) REFERENCES public.patients(id),
    CONSTRAINT family_relationships_relative_fkey FOREIGN KEY (relative_id) REFERENCES public.patients(id)
);
CREATE TABLE public.patients (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  abha_id text NOT NULL UNIQUE,
  name text NOT NULL,
  phone text,
  password text NOT NULL,
  registered_by uuid NOT NULL,
  CONSTRAINT patients_pkey PRIMARY KEY (id),
    CONSTRAINT patients_registered_by_fkey FOREIGN KEY (registered_by) REFERENCES public.pharmacists(id)
);
CREATE TABLE public.pharmacists (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  name text NOT NULL,
  username text NOT NULL UNIQUE,
  password text NOT NULL,
  phone text,
  license_number text NOT NULL UNIQUE,
  CONSTRAINT pharmacists_pkey PRIMARY KEY (id)
);
```


pharmacists
uuid id PK
text name
text license_number UK
admins
uuid id PK
text username UK
registers
patients
uuid id PK
text abha_id UK
text name
text phone
uuid registered_by FK
makes
receives
has
has relative
access_requests
uuid id PK
uuid pharmacist_id FK
uuid patient_id FK
text status
abdm_mock_records
text abha_id PK,FK
jsonb basic_health_details
jsonb pre_existing_conditions
jsonb medication_history
jsonb allergies
family_relationships
uuid id PK
uuid patient_id FK
uuid relative_id FK
text relationship_type

ER Diagram



Supabase DB Schema Diagram

6. Core Features

6.1 Brand-to-Generic Mapping

Most customers know medicine by brand name rather than generic name. The system bridges this gap by translating a brand such as Dolo or Crocin into its underlying salt, such as Paracetamol. This is essential because safety checks must operate on the chemical ingredient, not only on the market name printed on the box.

6.2 Personal Safety Engine

The safety engine compares the selected medicine against the patient's current medication list and health conditions. It checks for duplicate therapy, drug-drug interaction risks, and drug-to-condition conflicts. This allows the pharmacist to detect cases where a medicine could worsen an existing illness or clash with another active drug.

6.3 Clear Priority Alerts

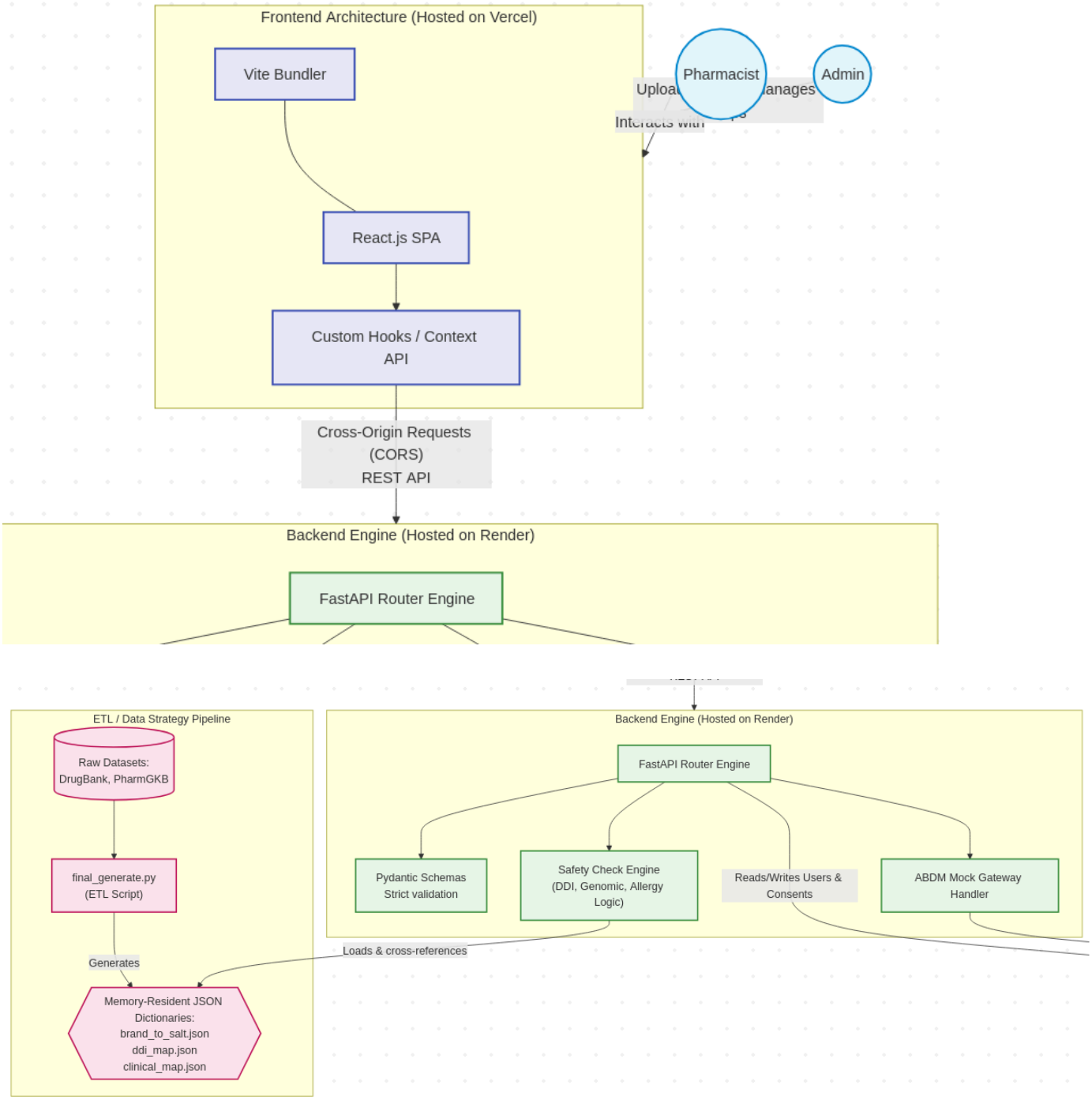
The interface classifies results into clearly visible categories so that the pharmacist can act quickly. Critical alerts are highlighted as urgent warnings, while lower-risk notes remain informational and less intrusive.

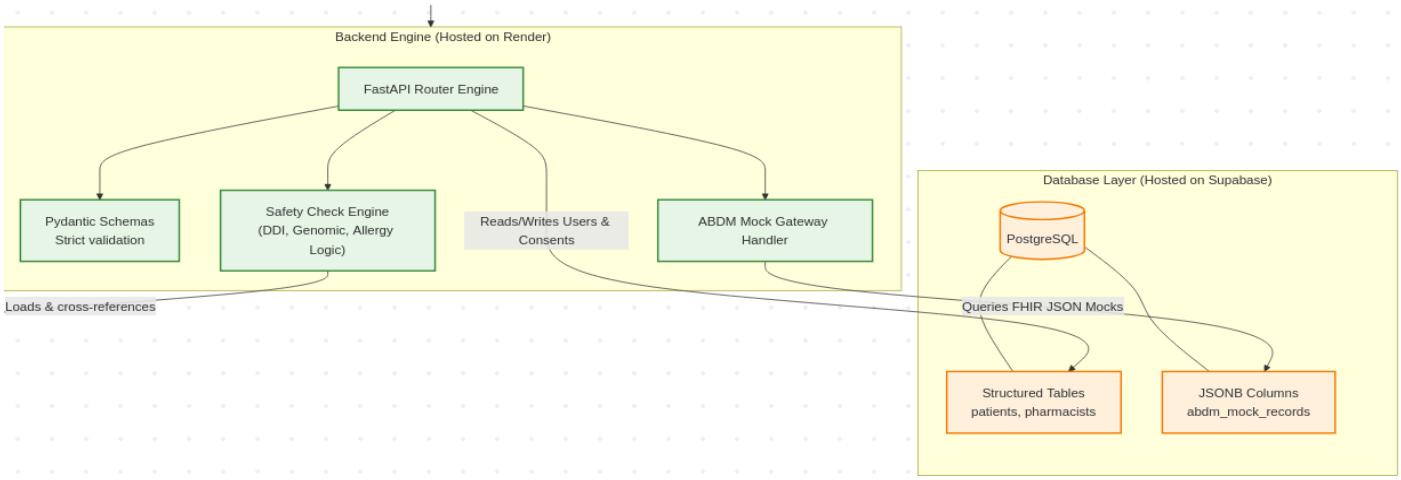
---

7. Solution Idea & Architecture

The project strongly separates concerns into a static UI layer (React/Vite) for highly responsive SPA mechanics, and a heavy computational API layer (FastAPI) for deterministic safety scoring.

1. Data ETL Pipeline: final_generate.py ingests raw datasets (DrugBank, PharmGKB) into optimized, memory-resident JSON dictionaries (brand_to_salt, ddi_map, clinical_map).
2. Step 1: Pharmacist issues an Access Request to a specific patient's ABHA ID.
3. Step 2: Patient grants consent, unlocking their ABDM Mock Records.
4. Step 3: Pharmacist inputs a brand name into the DashboardPanel.
5. Step 4: Frontend calls POST /api/check-drug.
6. Step 5: Backend resolves brand to salt and cross-references patient's active meds and genomics against the memory-resident JSON maps.
7. Step 6: Backend checks family_relationships for inherited risks.


Frontend Architecture (Hosted on Vercel)
Vite Bundler
React.js SPA
Custom Hooks / Context API
Pharmacist
Admin
Cross-Origin Requests (CORS)
REST API
Backend Engine (Hosted on Render)
FastAPI Router Engine
ETL / Data Strategy Pipeline
Raw Datasets: DrugBank, PharmGKB
final_generate.py (ETL Script)
Generates
Memory-Resident JSON Dictionaries: brand_to_salt.json ddi_map.json clinical_map.json
Backend Engine (Hosted on Render)
FastAPI Router Engine
Pydantic Schemas Strict validation
Safety Check Engine (DDI, Genomic, Allergy Logic)
Reads/Writes Users & Consents
ABDM Mock Gateway Handler
Loads & cross-references



---

8. Patient Medical History, ABDM Sandbox, and API Security

The project uses simulated ABDM-style records rather than live patient data. This is necessary because medical data is sensitive and the prototype is intended for controlled testing. The system follows the same general structure as the ABDM workflow, including consent-based access and FHIR-like medical record parsing.

In the current prototype, the pharmacist requests access using a patient identifier, the patient approves the request, and the backend then reads the available record to extract current medicines and health conditions. The logic is then applied to the medicine being checked.

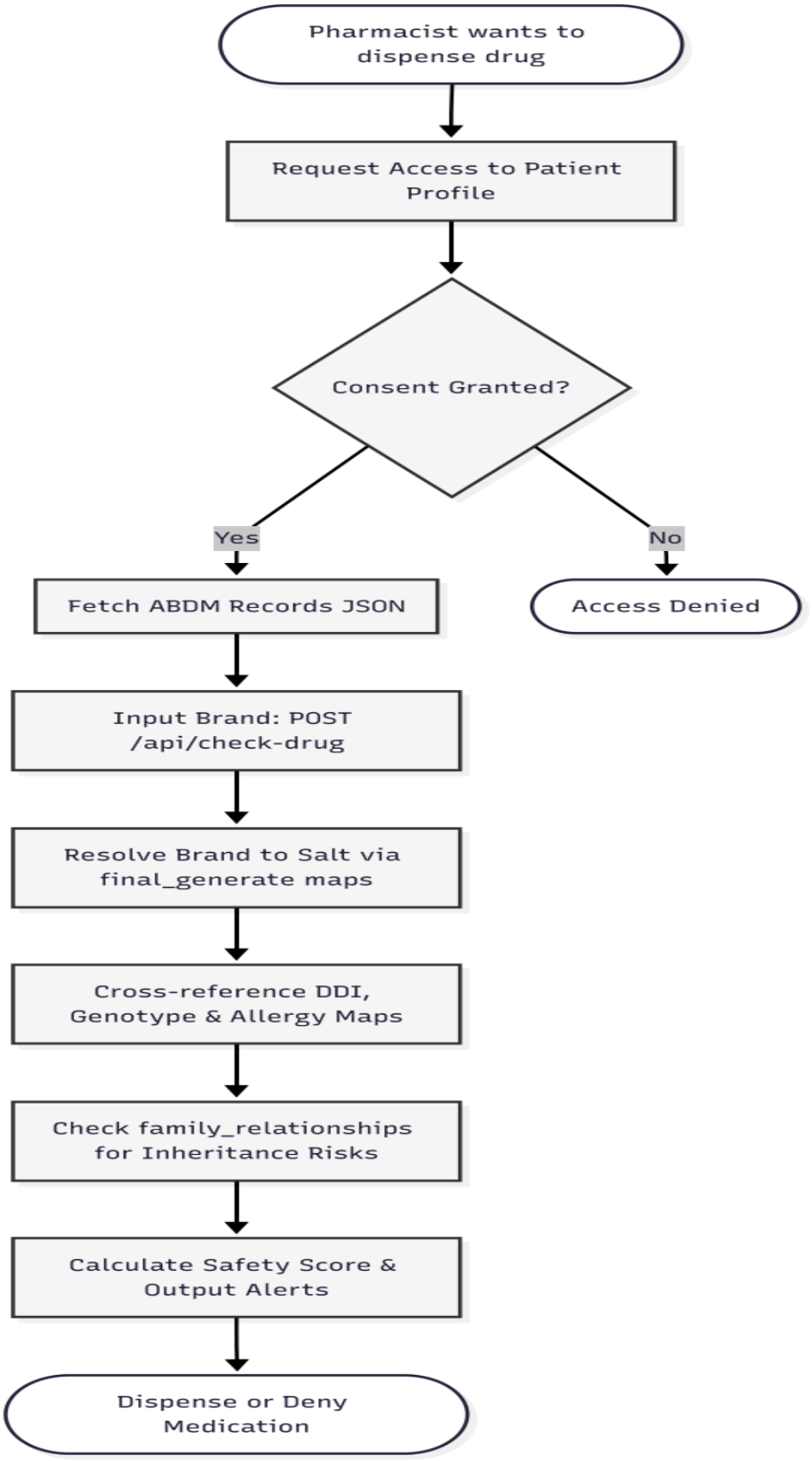
Security is supported at the database level through Supabase and PostgreSQL constraints. The database is configured so that public access is restricted and only the required read operations are exposed through the application layer.

---

9. Workflow

1. The pharmacist enters a medicine name into the dashboard.
2. The backend normalizes the input and resolves the brand to its generic salt.
3. The system fetches the patient's consent-approved medical data from the database.
4. The medicine is checked against current medication history, medical conditions, and interaction rules.
5. The backend evaluates severity and prepares the alert message.
6. If required, the system suggests a safer substitute from the available therapeutic set.

The workflow is kept fast and predictable so that the pharmacist can use it during a real counter transaction without noticeable delay.



Workflow Diagram

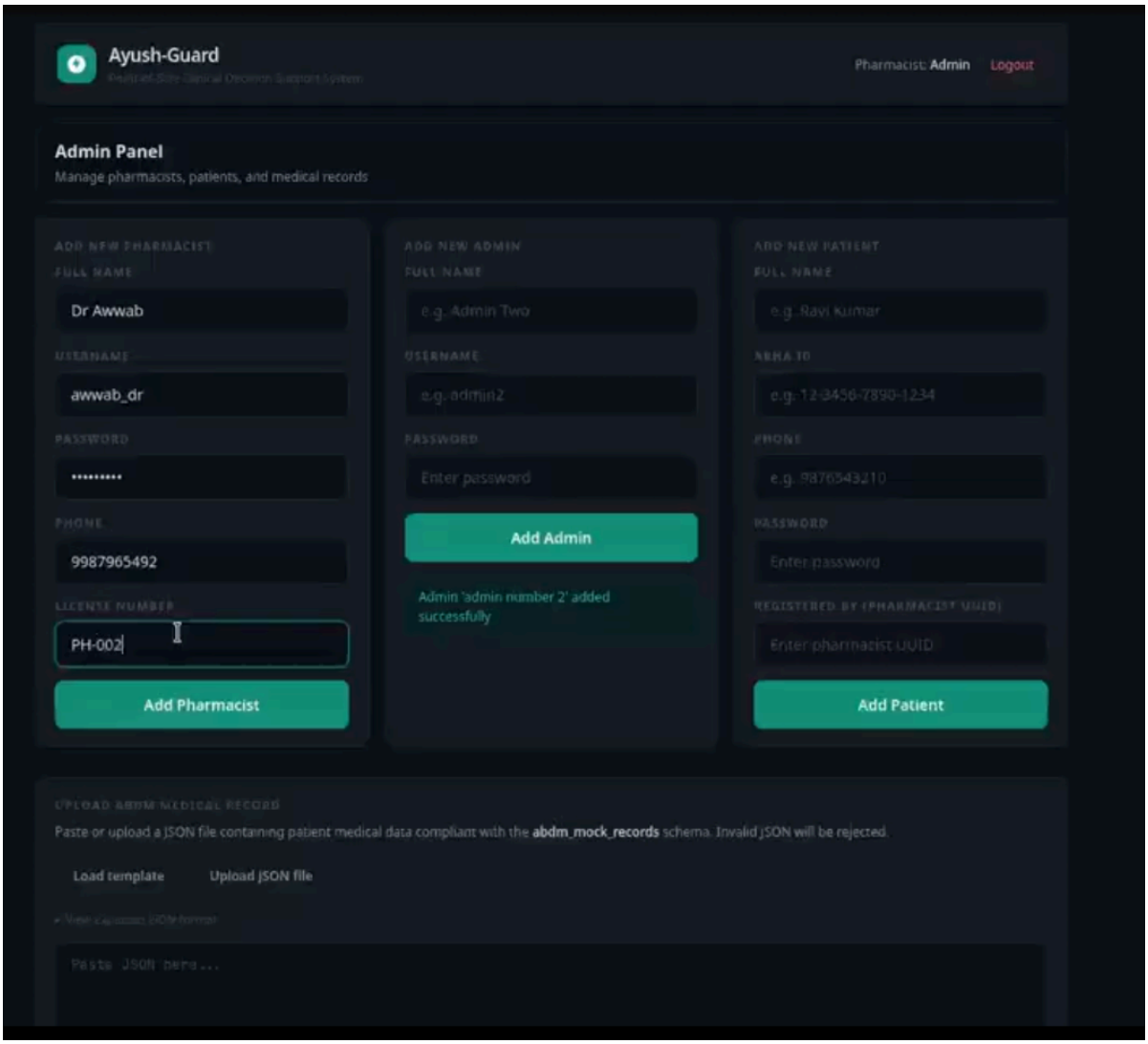
10. Technical Stack and Architecture Rationale

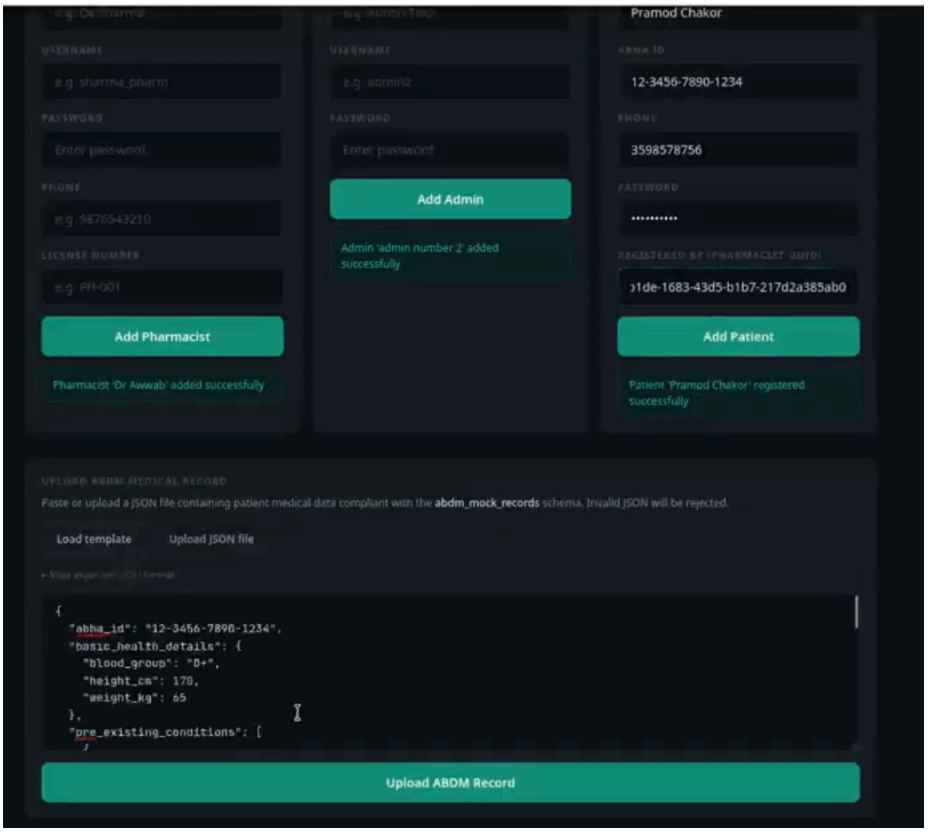
10.1 Frontend

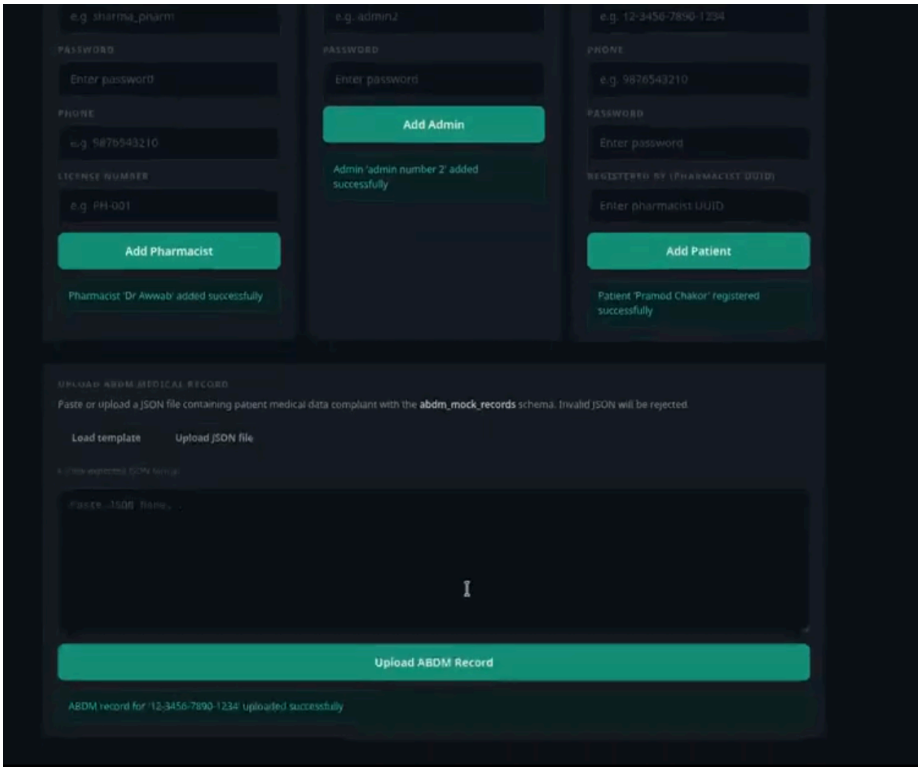
- **Implementation:** A lightweight, client-side web application.
- **Rationale:** Retail pharmacies require sub-second interaction times. React.js handles the complex state management required to simultaneously render the pharmacist's input, the fetched ABDM patient profile, and the 3-Tier Risk Assessment UI without page reloads. Tailwind CSS provides utility-first styling, ensuring the DOM remains lightweight and the dashboard is highly responsive across diverse pharmacy hardware.

Some Screenshots of the UI:

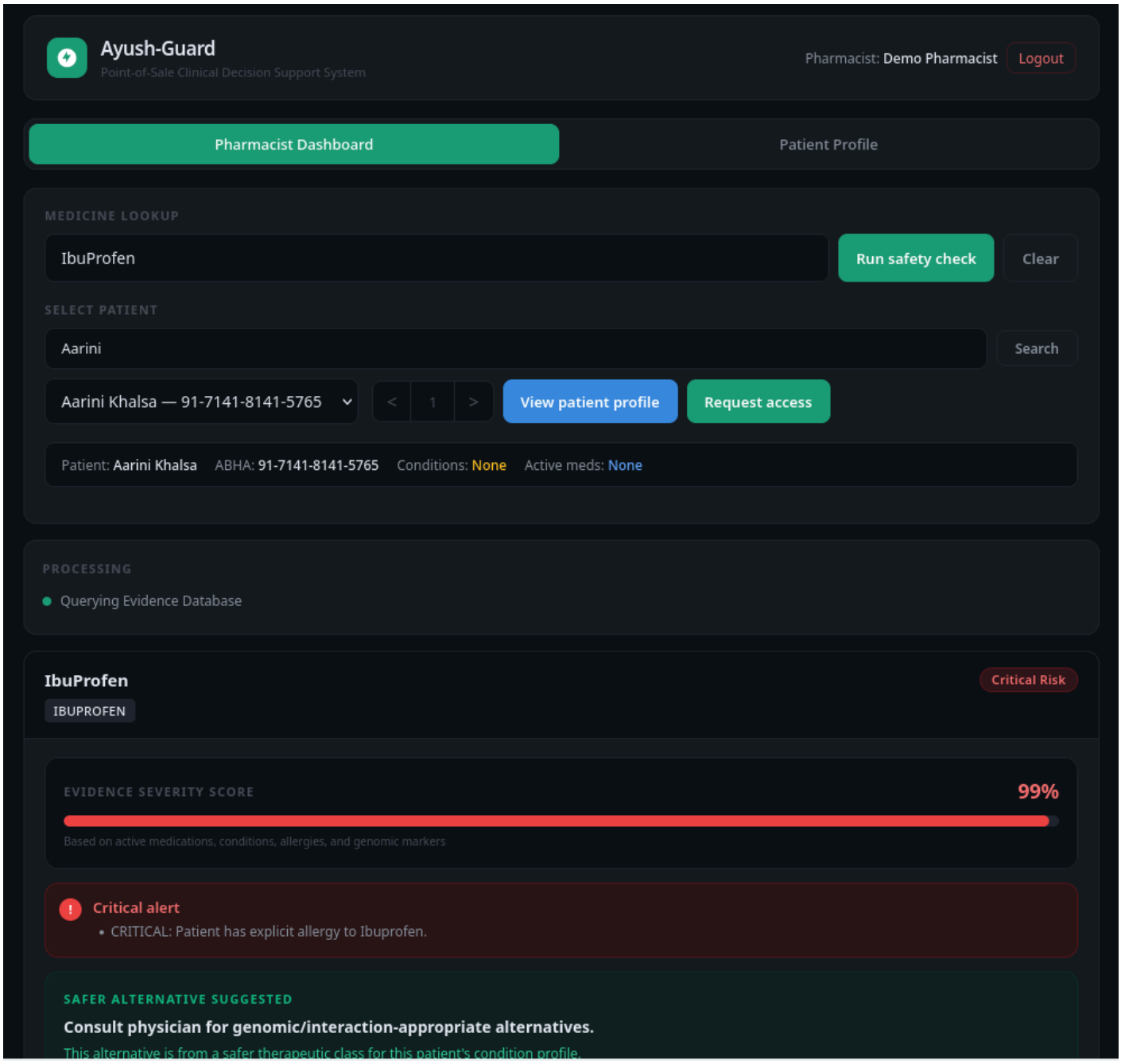
10.1.1 Admin Panel for adding new pharmacists/patients/admins:

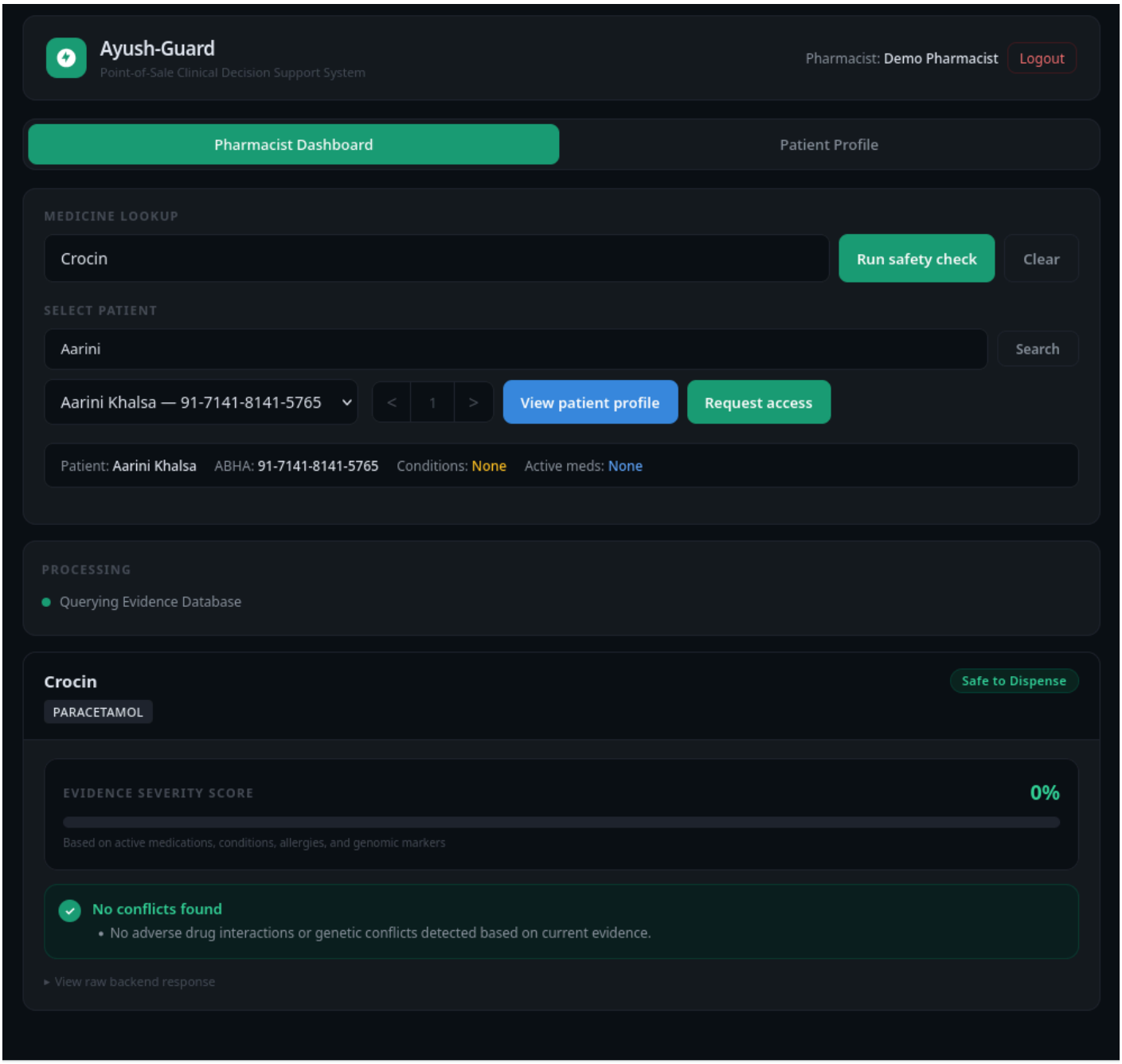


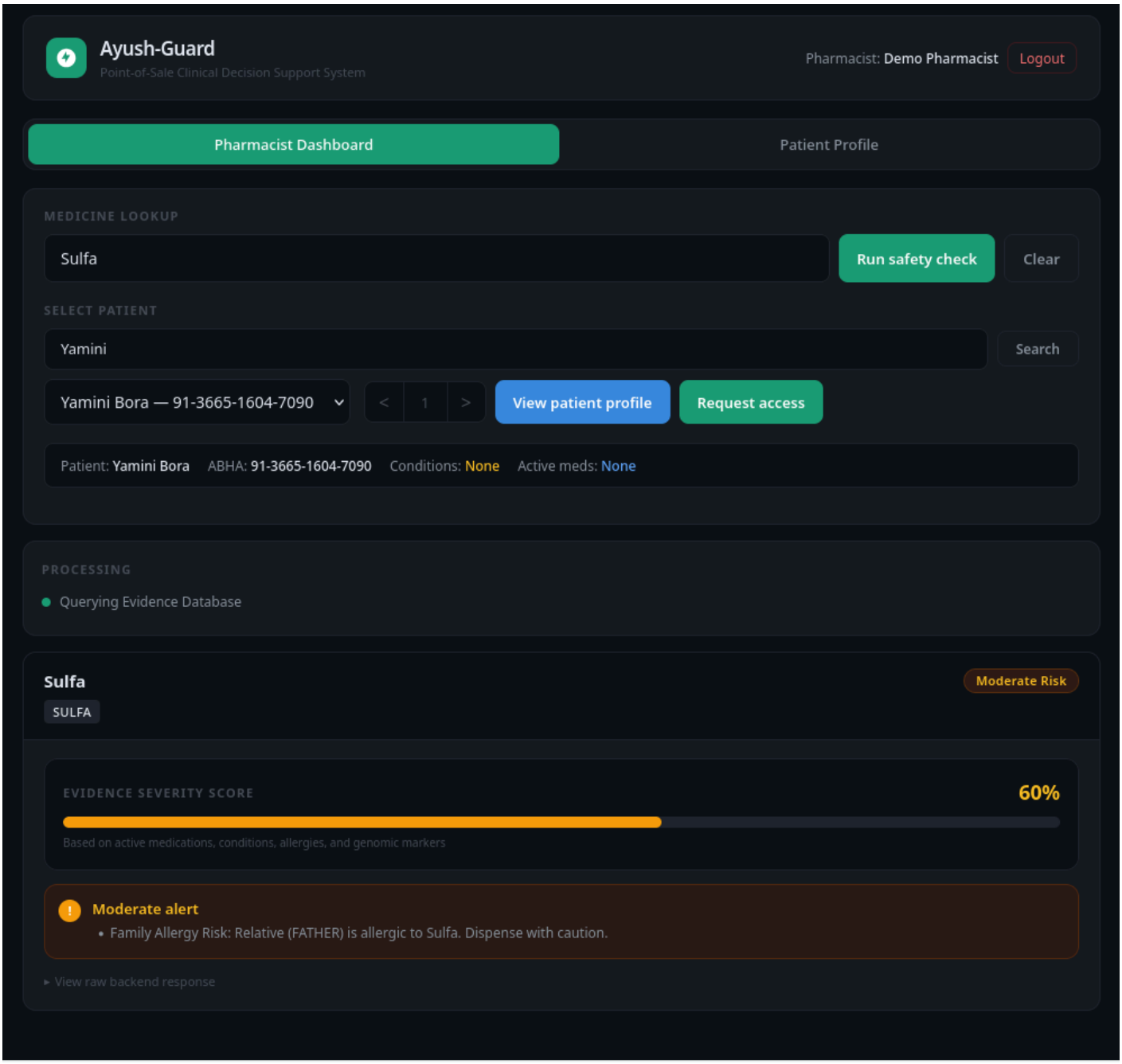

Pramod Chakor
12-3456-7890-1234
3598578756
Add Admin
Admin 'admin number 2' added successfully
Add Pharmacist
Pharmacist 'Dr Awwab' added successfully
Add Patient
Patient 'Pramod Chakor' registered successfully
Paste or upload a JSON file containing patient medical data compliant with the abdm_mock_records schema. Invalid JSON will be rejected.
Load template
Upload JSON file
{
"abha_id": "12-3456-7890-1234",
"basic_health_details": {
"blood_group": "O+",
"height_cm": 170,
"weight_kg": 65
},
"pre_existing_conditions": [
Upload ABDM Record


Add Admin
Admin 'admin number 2' added successfully
Add Pharmacist
Pharmacist 'Dr Awwab' added successfully
Add Patient
Patient 'Pramod Chakor' registered successfully
Load template
Upload JSON file
Upload ABDM Record
ABDM record for '12-3456-7890-1234' uploaded successfully

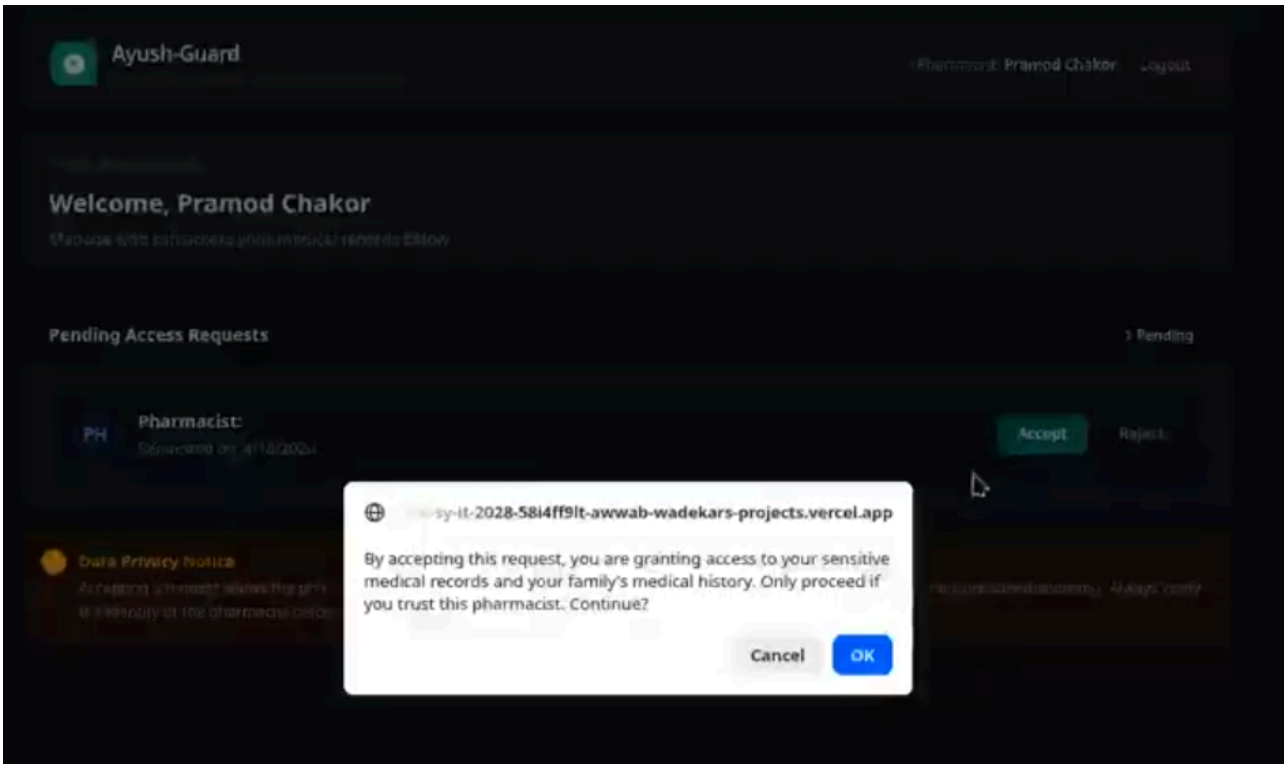
10.1.2 Pharmacist Panel for querying patient prescriptions

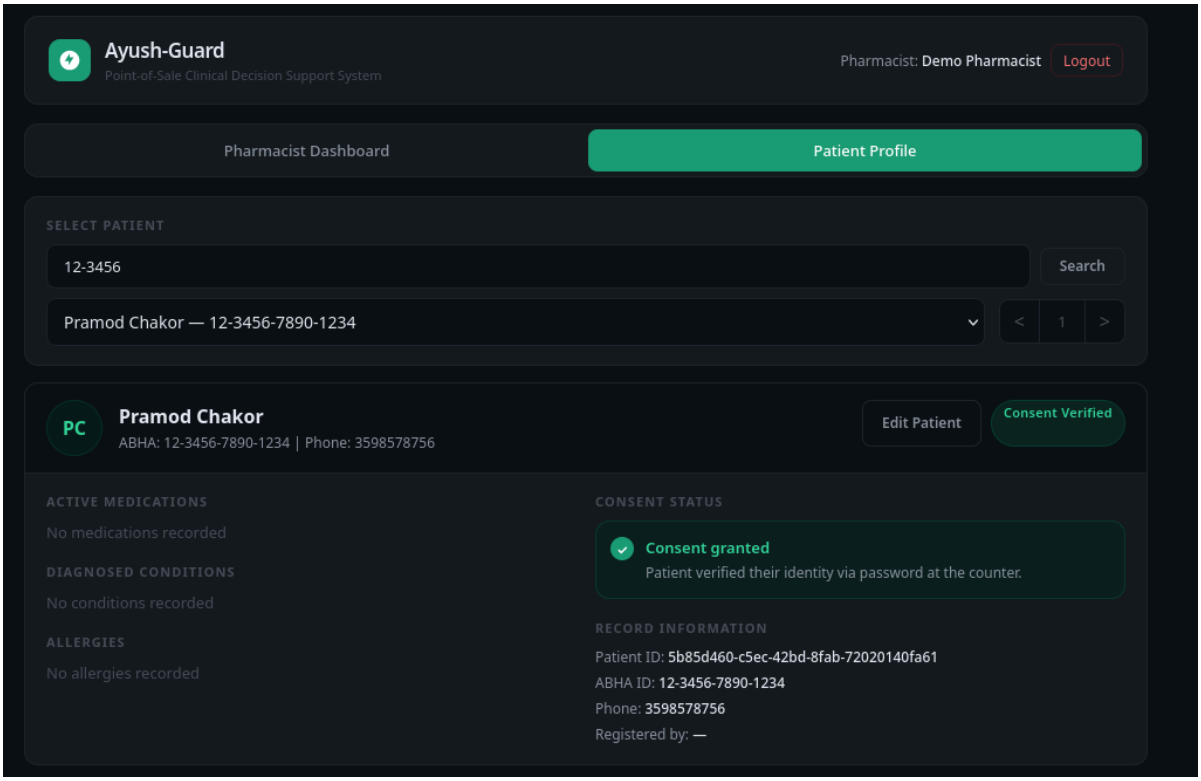



Ayush-Guard
Point-of-Sale Clinical Decision Support System
Pharmacist: Demo Pharmacist
Logout
Pharmacist Dashboard
Patient Profile
MEDICINE LOOKUP
Crocin
Run safety check
Clear
SELECT PATIENT
Aarini
Search
Aarini Khalsa — 91-7141-8141-5765
1
View patient profile
Request access
Patient: Aarini Khalsa
ABHA: 91-7141-8141-5765
Conditions: None
Active meds: None
PROCESSING
Querying Evidence Database
Crocin
PARACETAMOL
Safe to Dispense
EVIDENCE SEVERITY SCORE
0%
Based on active medications, conditions, allergies, and genomic markers
No conflicts found
• No adverse drug interactions or genetic conflicts detected based on current evidence.
View raw backend response


Ayush-Guard
Point-of-Sale Clinical Decision Support System
Pharmacist: Demo Pharmacist
Logout
Pharmacist Dashboard
Patient Profile
MEDICINE LOOKUP
Sulfa
Run safety check
Clear
SELECT PATIENT
Yamini
Search
Yamini Bora — 91-3665-1604-7090
1
View patient profile
Request access
Patient: Yamini Bora
ABHA: 91-3665-1604-7090
Conditions: None
Active meds: None
PROCESSING
Querying Evidence Database
Sulfa
SULFA
Moderate Risk
EVIDENCE SEVERITY SCORE
60%
Based on active medications, conditions, allergies, and genomic markers
Moderate alert
• Family Allergy Risk: Relative (FATHER) is allergic to Sulfa. Dispense with caution.
View raw backend response

10.1.3 Patient Panel and acceptance of request for data:

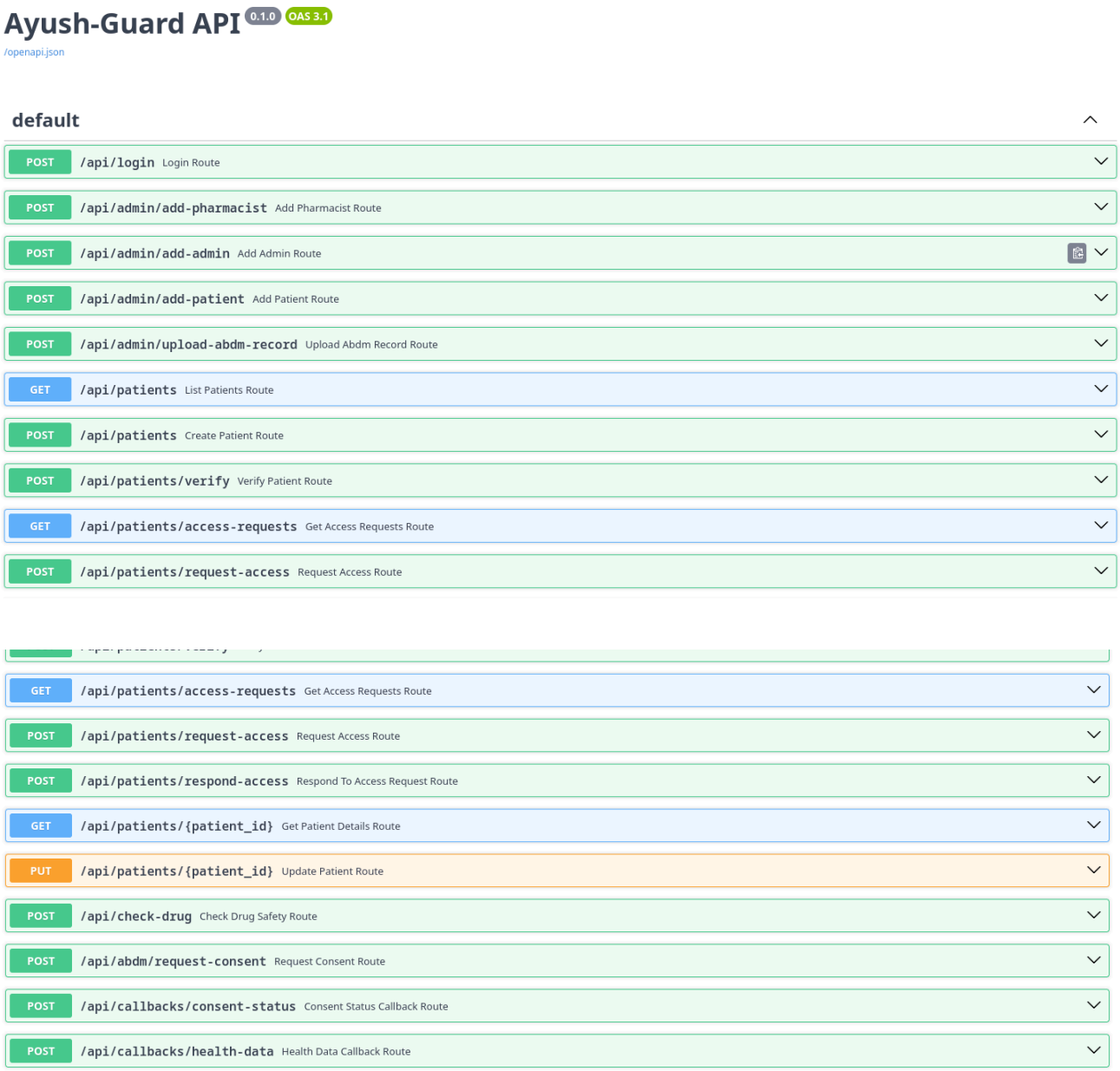




10.2 Backend

- **Implementation:** An asynchronous REST API acting as the central nervous system.
- **Rationale:** FastAPI was selected over traditional frameworks (like Django or Express) for two critical reasons. First, its native asynchronous support (`asyncio`) is mandatory for handling concurrent I/O operations—specifically, querying our Supabase database. Second, being Python-based, it allows us to run our heavy NLP and string-matching libraries (SciSpacy, FuzzyWuzzy) natively within the same server environment, eliminating the need for complex microservices.

Backend Routes (<backend_url>//docs)

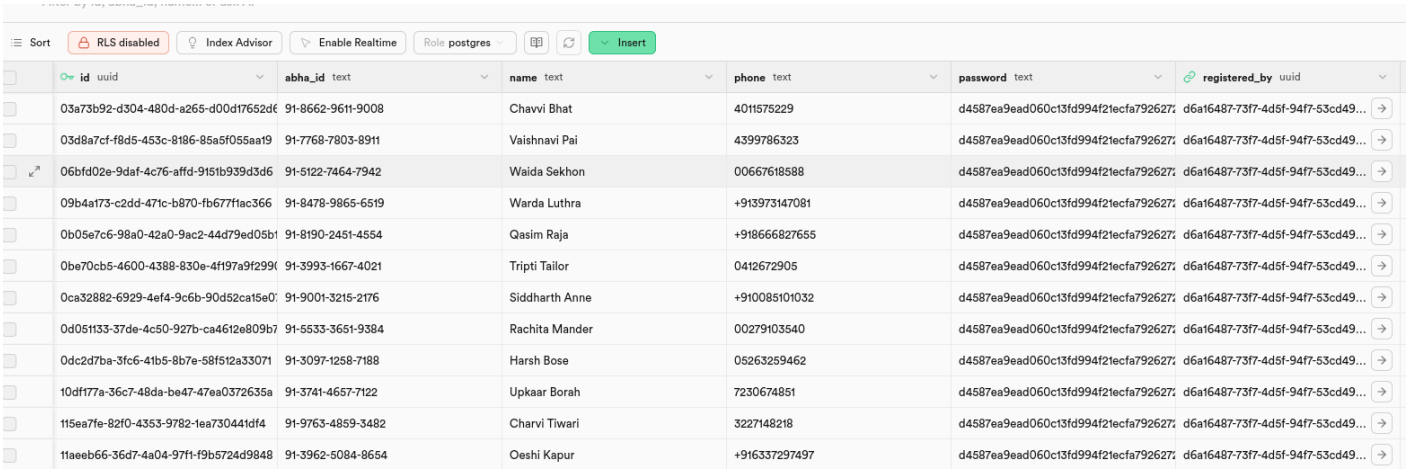


10.3 Database

Supabase provides a hosted PostgreSQL layer with structured tables and database management features. Using a relational database helps preserve data integrity and makes foreign key enforcement possible.

Some DB Screenshots

Patients Table



id uuid	abha_id text	name text	phone text	password text	registered_by uuid
03a73b92-d304-480d-a265-d00d17652d6	91-8662-9611-9008	Chavvi Bhat	4011575229	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
03d8a7cf-f8d5-453c-8186-85a5f055aa19	91-7768-7803-8911	Vaishnavi Pai	4399786323	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
06bfd02e-9daf-4c76-affd-9151b939d3d6	91-5122-7464-7942	Waida Sekhon	00667618588	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
09b4a173-c2dd-471c-b870-fb677f1ac366	91-8478-9865-6519	Warda Luthra	+913973147081	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
0b05e7c6-98a0-42a0-9ac2-44d79ed05b1	91-8190-2451-4554	Qasim Raja	+918666827655	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
0be70cb5-4600-4388-830e-4f197a9f299(	91-3993-1667-4021	Tripti Tailor	0412672905	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
0ca32882-6929-4ef4-9c6b-90d52ca15e0	91-9001-3215-2176	Siddharth Anne	+910085101032	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
0d051133-37de-4c50-927b-ca4612e809b7	91-5533-3651-9384	Rachita Mander	00279103540	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
0dc2d7ba-3fc6-41b5-8b7e-58f512a33071	91-3097-1258-7188	Harsh Bose	05263259462	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
10df177a-36c7-48da-be47-47ea0372635a	91-3741-4657-7122	Upkaar Borah	7230674851	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
115ea7fe-82f0-4353-9782-1ea730441df4	91-9763-4859-3482	Charvi Tiwari	3227148218	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...
11aeeb66-36d7-4a04-97f1-f9b5724d9848	91-3962-5084-8654	Oeshi Kapur	+916337297497	d4587ea9ead060c13fd994f21ecfa792627	d6a16487-73f7-4d5f-94f7-53cd49...

Pharmacists Table

id u...	name text	username t...	password te	phone text	license_number
d6a16487-73f7-4	Demo Pharmacist	pharma1	1c7d7f2a7668a1de	NULL	PH-DEMO-001
ffb0b1de-1683-4	Dr Awwab	awwab_dr	1c7d7f2a7668a1de	9987965492	PH-002

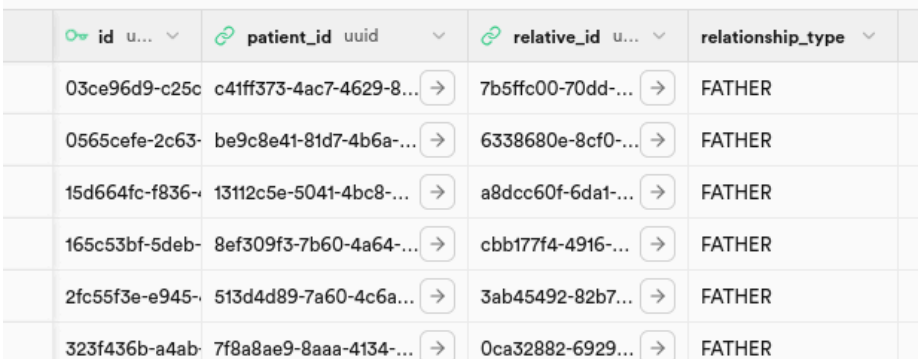
Admins Table

id u...	name text	username text	password
76293386-2c7b	Admin	admin	240be518fabd2
f3278adb-8cd0-	admin number 2	admin2	240be518fabd2

Access Requests Table

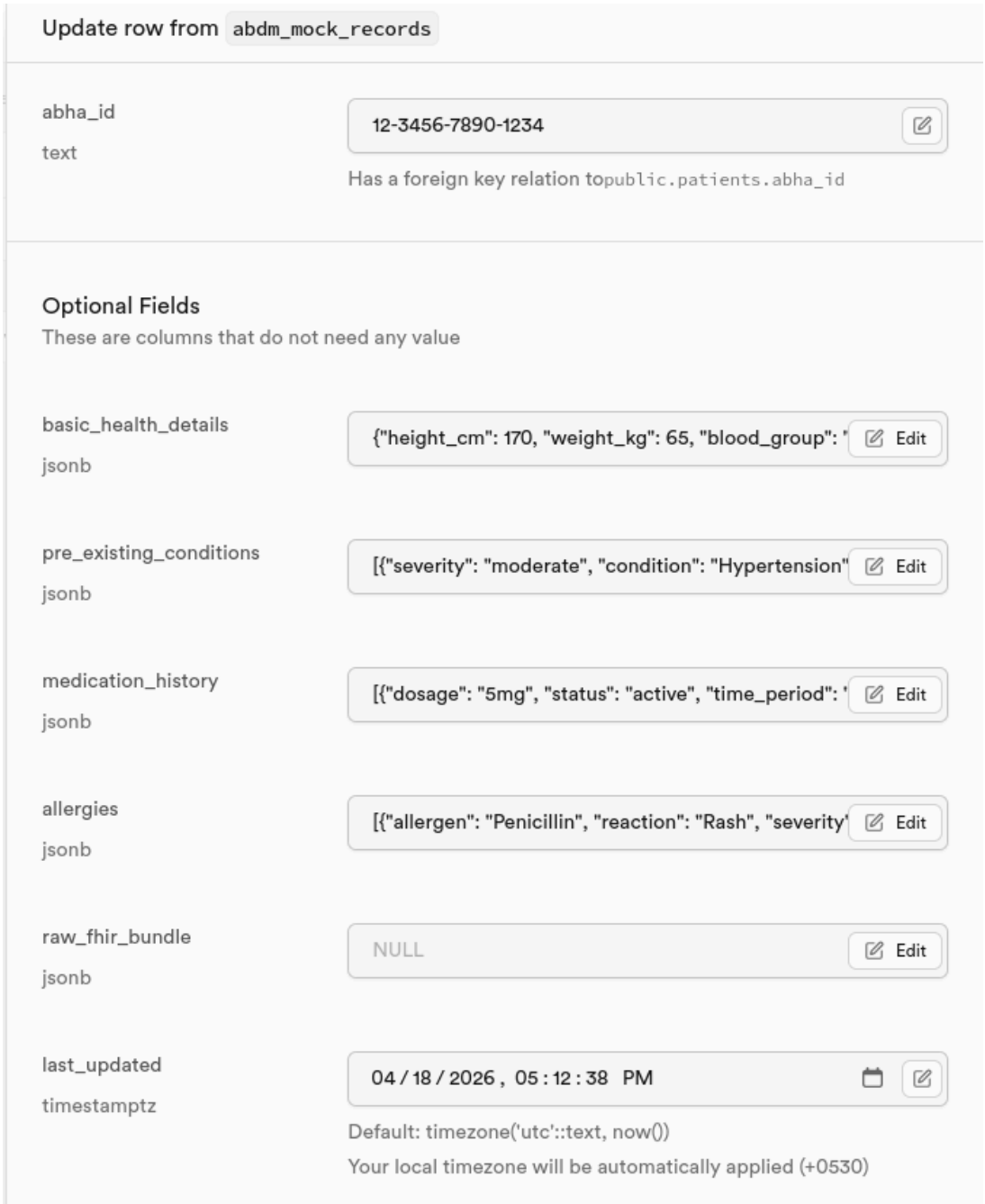
id u...	pharmacist_i	patient	status t...	created_at ti...	updated_at ti
00690c3f-695a	d6a16487-73f...	731fce0...	ACCEPTED	2026-04-17 09:02:28	2026-04-17 09:02::
0215080c-4932	d6a16487-73f...	25a65d...	ACCEPTED	2026-04-17 09:02:28	2026-04-17 09:02::
03ceed17-b834-	d6a16487-73f...	3651186...	ACCEPTED	2026-04-17 09:02:28	2026-04-17 09:02::

Family Relations Table



id u...	patient_id uuid	relative_id u...	relationship_type
03ce96d9-c25c	c41ff373-4ac7-4629-8...	7b5ffc00-70dd-...	FATHER
0565cefe-2c63-	be9c8e41-81d7-4b6a-...	6338680e-8cf0-...	FATHER
15d664fc-f836-	13112c5e-5041-4bc8-...	a8dcc60f-6da1-...	FATHER
165c53bf-5deb-	8ef309f3-7b60-4a64-...	cbb177f4-4916-...	FATHER
2fc55f3e-e945-	513d4d89-7a60-4c6a...	3ab45492-82b7...	FATHER
323f436b-a4ab	7f8a8ae9-8aaa-4134-...	0ca32882-6929...	FATHER

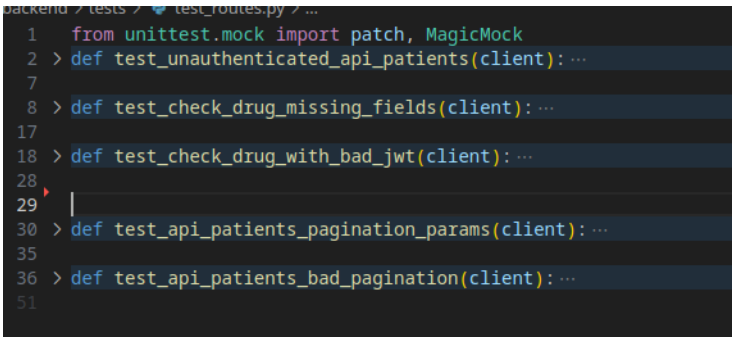
ABDM Record Table : Example Row

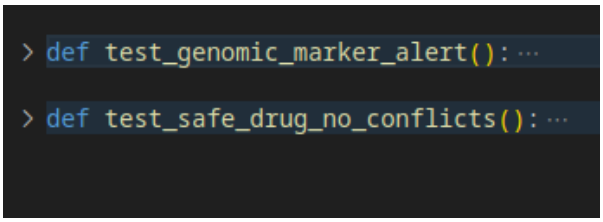


11. Testing and Constraints

Testing was carried out at the backend level with a mock DB setup to verify that the API routes and safety checks behave correctly. The backend exposes interactive API documentation through FastAPI's Swagger interface at the /docs endpoint, which makes route inspection and manual testing easier during development.

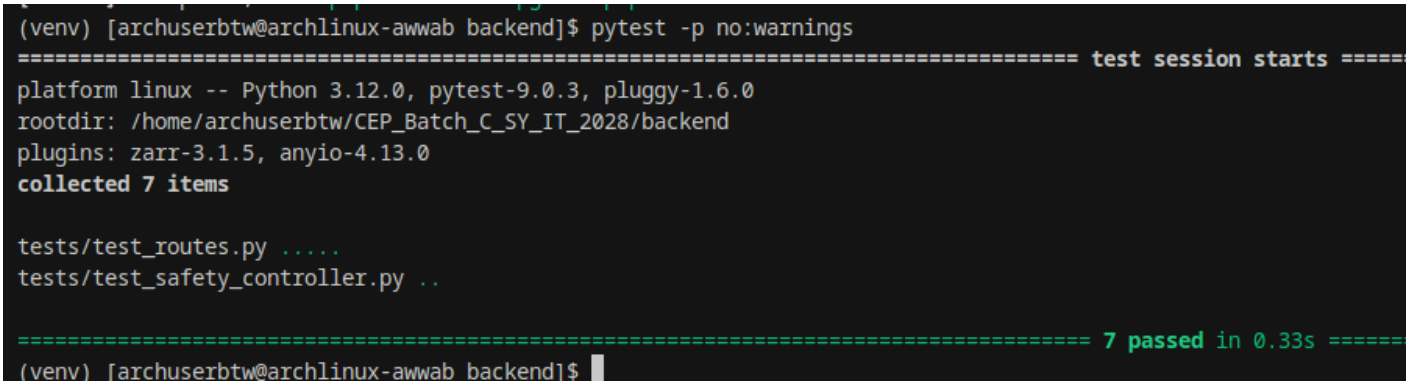
Functionalities tested:





The following command is used to run the test suite after activating the backend virtual environment:

pytest -p no:warnings



Network Tab used for testing routes:

Status	Method	Domain	File	Initiator	Type	Transferred	Size
	OPTIONS	cep-batch-c-sy-it-2028.onrender.com	login	fetch			
200	POST	cep-batch-c-sy-it-2028.onrender.com	login	fetch	json	641 B	136 B
	OPTIONS	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20	fetch			
200	GET	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20	fetch	json	2.33 kB	12.56 kB
200	GET	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20	index-D0vlmD09.js:49 (fetch)	json	2.33 kB	12.56 kB
200	GET	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20	fetch	json	2.33 kB	12.56 kB
200	GET	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20	fetch	json	2.33 kB	12.56 kB
200	GET	cep-batch-c-sy-it-2028.onrender.com	ac60ca51-49dc-4ae1-a1df-79c14969eb15	index-D0vlmD09.js:55 (fetch)	json	960 B	645 B
200	OPTIONS	cep-batch-c-sy-it-2028.onrender.com	ac60ca51-49dc-4ae1-a1df-79c14969eb15	fetch	plain	698 B	2 B
200	GET	cep-batch-c-sy-it-2028.onrender.com	ac60ca51-49dc-4ae1-a1df-79c14969eb15	index-D0vlmD09.js:55 (fetch)	json	960 B	645 B
200	GET	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20	index-D0vlmD09.js:55 (fetch)	json	2.33 kB	12.56 kB
200	GET	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20&search=12-3456	index-D0vlmD09.js:55 (fetch)	json	661 B	157 B
200	OPTIONS	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20&search=12-3456	fetch	plain	698 B	2 B
200	GET	cep-batch-c-sy-it-2028.onrender.com	5b85d460-c5ec-42bd-8fab-72020140fa61	index-D0vlmD09.js:55 (fetch)	json	683 B	188 B
200	OPTIONS	cep-batch-c-sy-it-2028.onrender.com	5b85d460-c5ec-42bd-8fab-72020140fa61	fetch	plain	698 B	2 B
200	GET	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20	index-D0vlmD09.js:49 (fetch)	json	2.33 kB	12.56 kB
200	GET	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20&search=Aarini	index-D0vlmD09.js:49 (fetch)	json	1.01 kB	710 B
200	OPTIONS	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20&search=Aarini	fetch	plain	698 B	2 B
200	POST	cep-batch-c-sy-it-2028.onrender.com	check-drug	index-D0vlmD09.js:49 (fetch)	json	692 B	211 B
200	OPTIONS	cep-batch-c-sy-it-2028.onrender.com	check-drug	fetch	plain	711 B	2 B
200	GET	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20&search=tanay	index-D0vlmD09.js:49 (fetch)	json	1.04 kB	1.26 kB
200	OPTIONS	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20&search=tanay	fetch	plain	698 B	2 B
200	POST	cep-batch-c-sy-it-2028.onrender.com	check-drug	index-D0vlmD09.js:49 (fetch)	json	753 B	304 B
200	GET	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20&search=Gunbir	index-D0vlmD09.js:49 (fetch)	json	1.01 kB	719 B
200	OPTIONS	cep-batch-c-sy-it-2028.onrender.com	patients?page=1&limit=20&search=Gunbir	fetch	plain	698 B	2 B
200	POST	cep-batch-c-sy-it-2028.onrender.com	check-drug	index-D0vlmD09.js:49 (fetch)	json	739 B	278 B

Pagination was added when loading the patient list on the pharmacist panel. This avoids sending too many records in a single response and reduces the chance of backend slowdown or crash during bulk loading.

The main constraints of the prototype are the use of sandbox or mock medical data instead of real live records, the dependency on valid foreign keys for relational integrity, and the fact that the system is designed as a controlled demonstration rather than a production clinical deployment.

- Pagination prevents excessive load while fetching patient records.
- Foreign key constraints ensure only valid linked records are stored.
- Sandbox data keeps the prototype safe for academic use.
- The /docs endpoint is used for route inspection and testing.
- Backend checks are validated through pytest-based testing.

12. Challenges Faced:-

- **Lack of Sensitive Clinical Data:** Due to privacy constraints, real patient datasets were unattainable. This occasionally caused the ML/NLP layers to hallucinate edge cases, forcing reliance on structured, mock demo injections to maintain deterministic safety scoring.
- **CORS Debugging:** Managing strict Cross-Origin Resource Sharing (CORS) rules between the isolated Vercel frontend and the Render-hosted backend during integration.
- **Data Parsing & Formats:** Handling semi-structured clinical JSON dumps required heavy usage of Pydantic to strictly enforce data validation across API boundaries.
- **Performance at Scale:** Fetching bulk user records strained network resources, requiring the implementation of backend pagination for efficient data retrieval in the GET /api/patients route.
- **Asynchronous Testing:** Ensuring that complex chained API endpoints evaluated safely under load necessitated rigorous Pytest implementations

---

13. Future Improvements

- **Real ABDM Integration:** Swapping mocked JSON records for active HTTPS requests to the National Health Authority (NHA) gateway, decrypting FHIR JSONs via Diffie-Hellman keys.
- **Deep Learning Embeddings:** Upgrading text-matching to use a medical BERT model (e.g., ClinicalBERT) to identify semantic similarities within unstructured physician notes.
- **Advanced State Management:** Utilizing React Query or Redux to cache patient histories locally, drastically reducing redundant API trips.
- **Dosage-Specific Risk Calculation:** Upgrading the ML engine to require drug strength arrays, distinguishing between safe low doses and toxic thresholds.
- **Safety Swap Suggestions:** A planned enhancement to Ayush-Guard involves extending the conflict detection system to include proactive therapeutic substitution suggestions. When a high-risk interaction or contraindication is identified, the system would not only flag the concern but also recommend a safer alternative from within the same treatment category.

---

14. Conclusion

Ayush-Guard demonstrates how a clinical decision support system can help pharmacists make safer medicine choices at the point of sale. By combining Supabase-based storage, FastAPI backend logic, and structured safety rules, the project creates a practical framework for checking medicine suitability against patient history. The prototype shows that a clean database design, constraint enforcement, and careful workflow planning can significantly improve medication safety in a pharmacy setting.

Project Deliverables

[Drive Link: Demo Video]
[https://drive.google.com/drive/folders/1tY_klodFTW-LLNPZm9NSv7Vj5MuLFM_M?usp=drive_link]

Hosted Site: https://cep-batch-c-sy-it-2028.vercel.app/

Github Repo: https://github.com/Once-1296/CEP_Batch_C_SY_IT_2028